

UNIVERSIDAD PRIVADA “FRANZ TAMAYO”

Facultad de Ingeniería



“Evaluación Procesual”

INTEGRANTES:

- Aldo Michael Figueredo Salvatierra

MATERIA:

- Ingeniería de software II

DOCENTE:

- Ing. Alvaro Gabriel Coronel Centellas

LA PAZ – BOLIVIA

2025

1. Resumen Ejecutivo

El Problema

ECOBOL (Empresa de Correos de Bolivia) opera una red de 9 sucursales a nivel nacional, pero hasta ahora gestionaba sus envíos postales de forma mayoritariamente manual. Esta situación generaba problemas concretos: errores frecuentes en el seguimiento de paquetes, demoras en la actualización de estados, dificultad para generar reportes gerenciales y ausencia de un sistema de roles que diferenciara las responsabilidades entre administradores y empleados. La falta de trazabilidad digital no solo afectaba la eficiencia operativa interna, sino también la experiencia de los clientes, quienes no podían consultar el estado de sus envíos de manera autónoma.

La Solución

Se desarrolló un sistema web completo compuesto por tres capas principales: una API REST construida en Node.js con Express, una interfaz de usuario tipo SPA (Single Page Application) en React con Vite, y una base de datos relacional PostgreSQL. El sistema cubre todo el ciclo de vida de un envío postal: desde su registro con generación automática de un código único de rastreo, pasando por la actualización de estados en tiempo real, hasta la consulta pública por parte del destinatario. Además, incluye un módulo de reportes exportables en formato CSV, un panel gerencial con gráficos estadísticos, y gestión diferenciada de usuarios según su rol. Todo el sistema se despliega mediante contenedores Docker, lo que simplifica tanto la instalación como el mantenimiento.

2. Análisis y Requisitos

Requisitos Funcionales

El sistema debe cumplir con las siguientes funcionalidades principales:

- Registro de envíos: los empleados pueden crear nuevos envíos indicando remitente, destinatario, sucursal de origen, sucursal de destino y tipo de servicio postal.
- Generación de códigos de rastreo únicos con el formato ECO-YYYYMMDD-NNNNN, asegurando la identificación inequívoca de cada envío.
- Actualización de estados a lo largo del ciclo de vida del envío, siguiendo la secuencia: Registrado, En tránsito, En sucursal destino y Entregado.
- Portal de rastreo público que permite a cualquier persona consultar el estado de un envío ingresando su código, sin necesidad de iniciar sesión.
- Gestión de usuarios con soporte para dos roles diferenciados: administrador y empleado, con permisos distintos en cada caso.
- Gestión de sucursales: el administrador puede crear, editar y administrar las 9 sucursales del país.
- Panel gerencial con estadísticas visuales que incluyen gráficos de envíos por estado y seguimiento de posibles retrasos.
- Exportación de reportes en formato CSV con filtros por fecha, sucursal y estado, para facilitar el análisis gerencial externo.
- Autenticación segura mediante tokens JWT, que protege las rutas privadas del sistema.
- Historial de estados por envío, con registro de cada cambio junto a su fecha y hora exacta.
- Validación de datos en todos los formularios del sistema, garantizando la integridad de la información ingresada.
- Interfaz responsive adaptada a distintos tamaños de pantalla, incluyendo dispositivos móviles y de escritorio.

Requisitos No Funcionales

Adicionalmente, el sistema debe satisfacer los siguientes criterios de calidad:

- Rendimiento: el tiempo de respuesta de la API no debe superar los 500 ms para las operaciones más frecuentes.
- Seguridad: uso de autenticación JWT, validación exhaustiva de entradas y protección contra inyección SQL mediante consultas parametrizadas.
- Disponibilidad: el sistema debe mantenerse operativo con un uptime del 99%, garantizado mediante contenedores Docker con política de reinicio automático.
- Mantenibilidad: el código debe estar correctamente formateado, organizado de forma modular y acompañado de documentación suficiente para facilitar futuras intervenciones.
- Compatibilidad: la interfaz debe funcionar correctamente en los navegadores modernos más utilizados, incluyendo Chrome, Firefox y Edge.
- Escalabilidad: la arquitectura debe permitir escalar componentes de forma independiente ante un eventual crecimiento en el volumen de operaciones.

3. Modelos UML

Diagrama de Casos de Uso

Actores principales: Empleado, Administrador y Usuario Público. Los casos de uso incluyen: iniciar sesión, registrar envío, actualizar estado, consultar envío, exportar reporte, gestionar usuarios y gestionar sucursales.

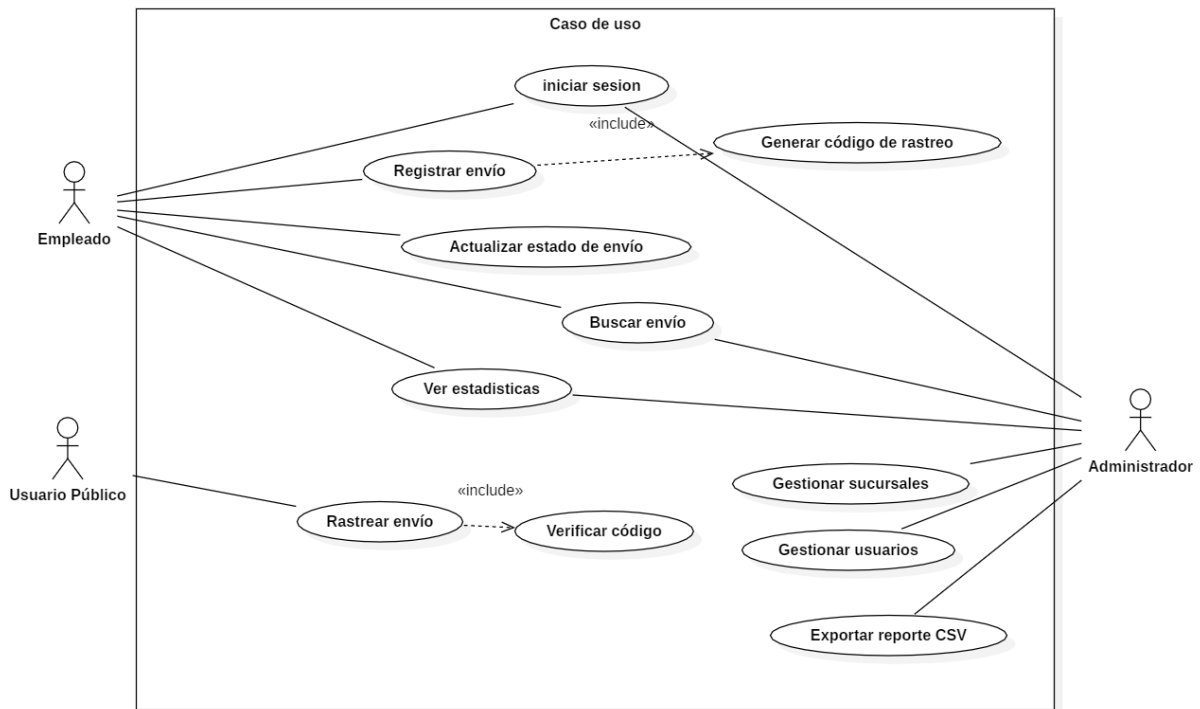
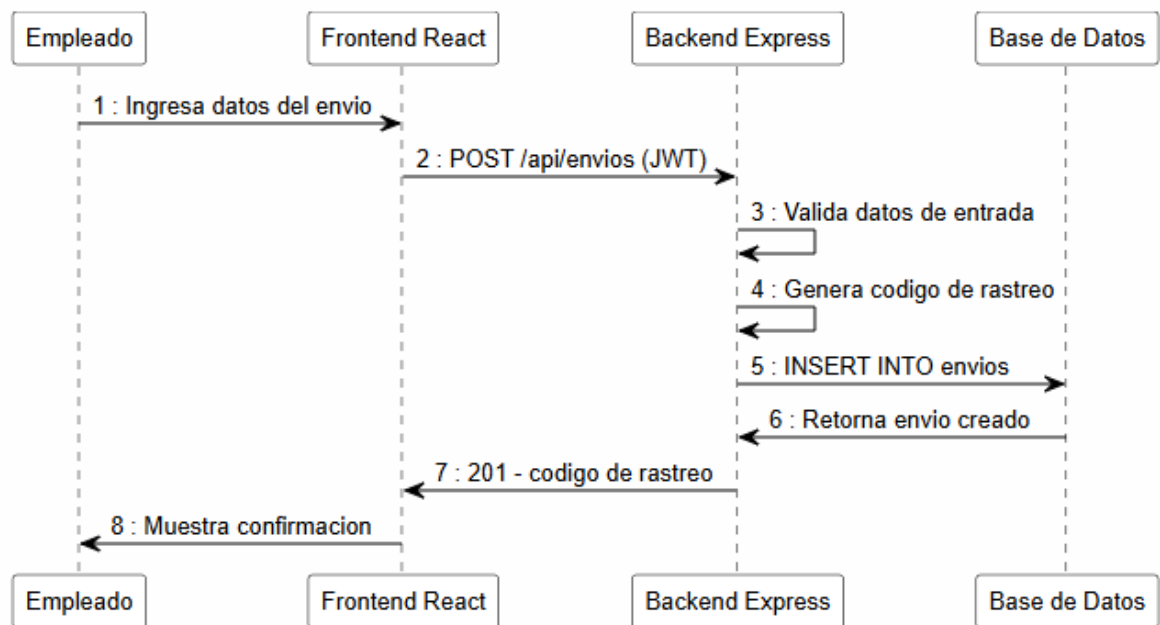
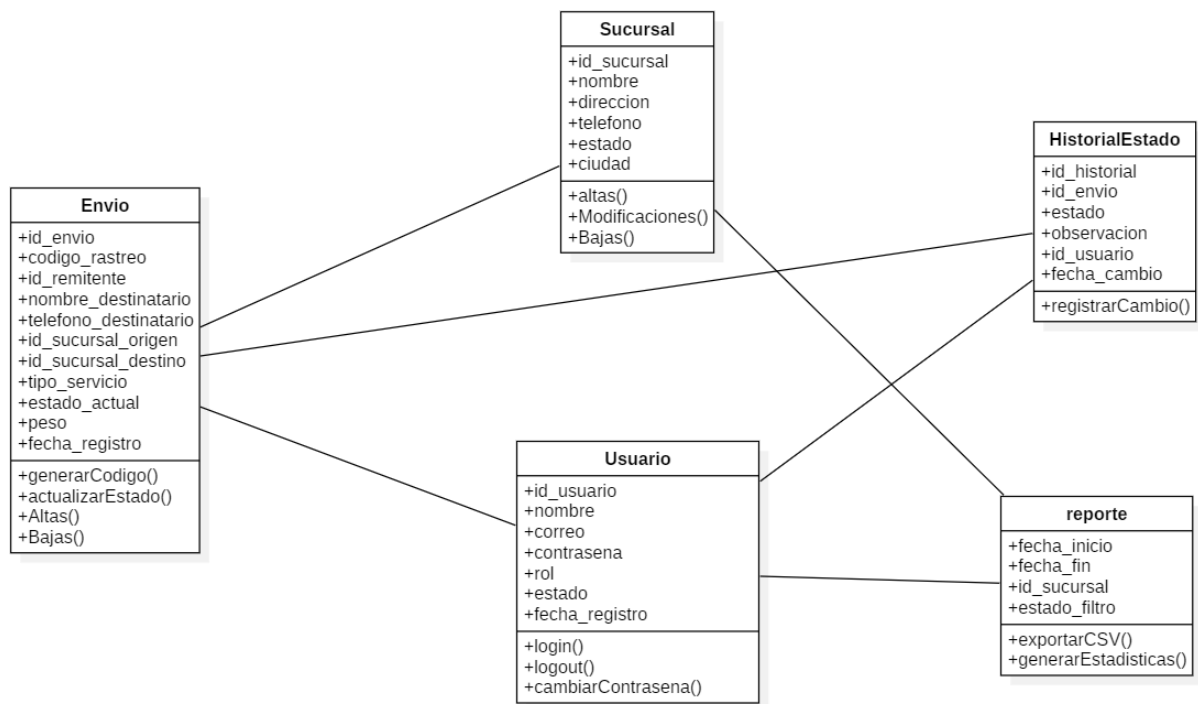


Diagrama de Clases

Las clases principales del sistema son: Usuario, Envio, Sucursal, EstadoEnvio e HistorialEstado. Se detallan sus atributos, métodos y las relaciones de asociación y herencia entre ellas.



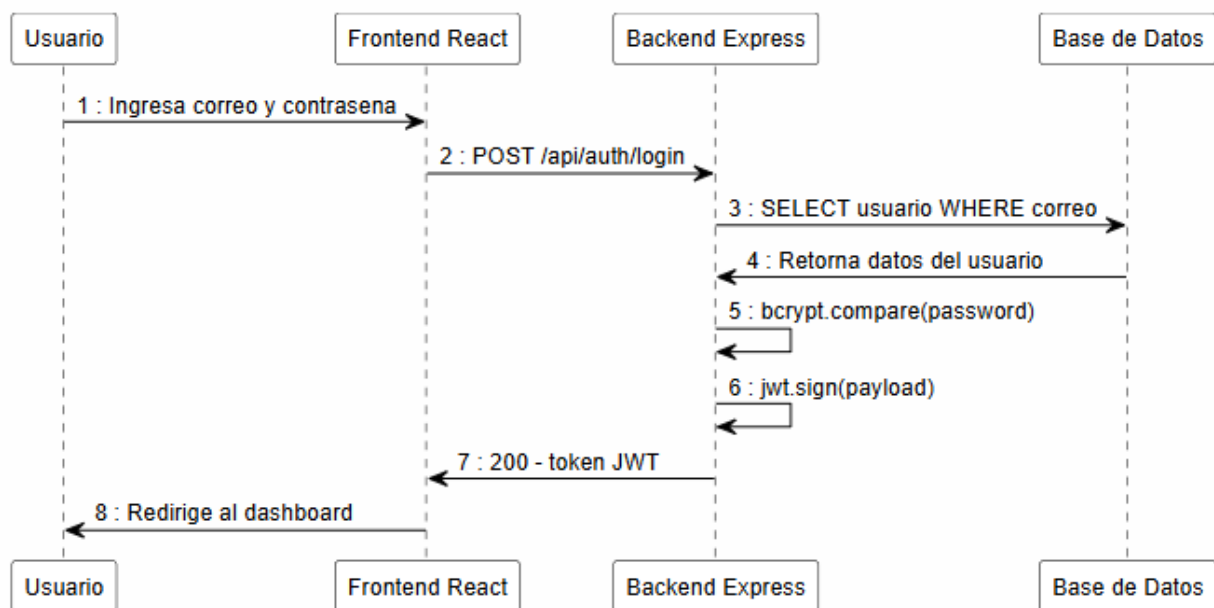
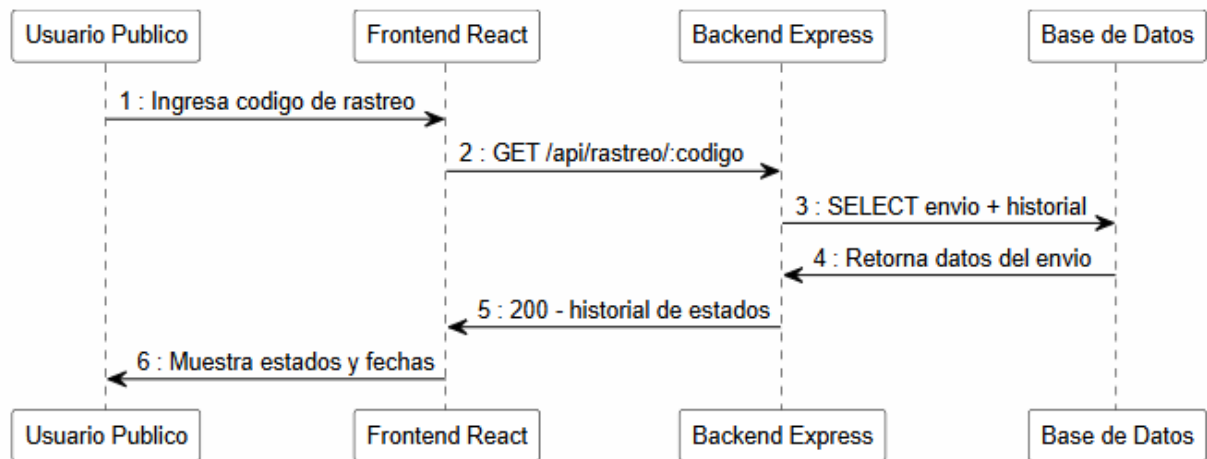
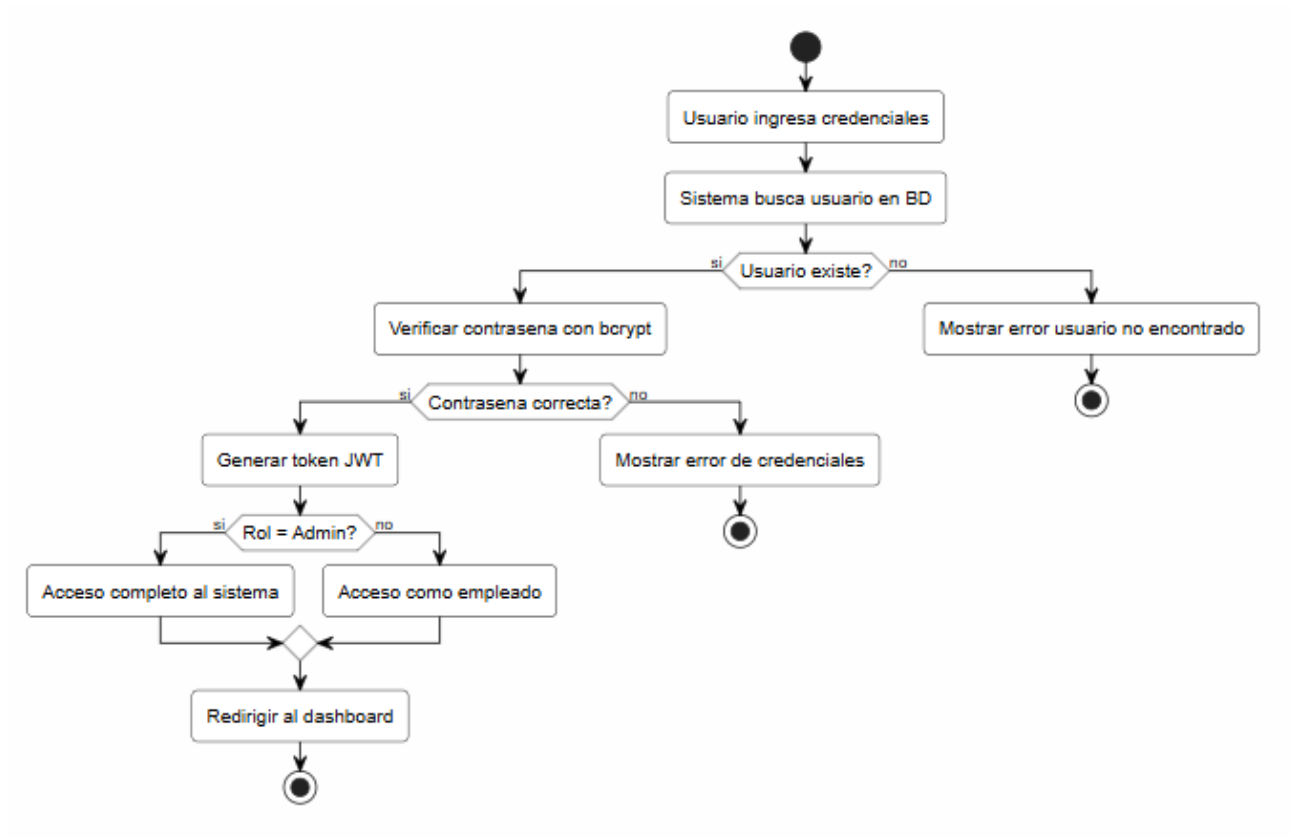


Diagrama de Actividades



4. Arquitectura de Software

Estilo Adoptado

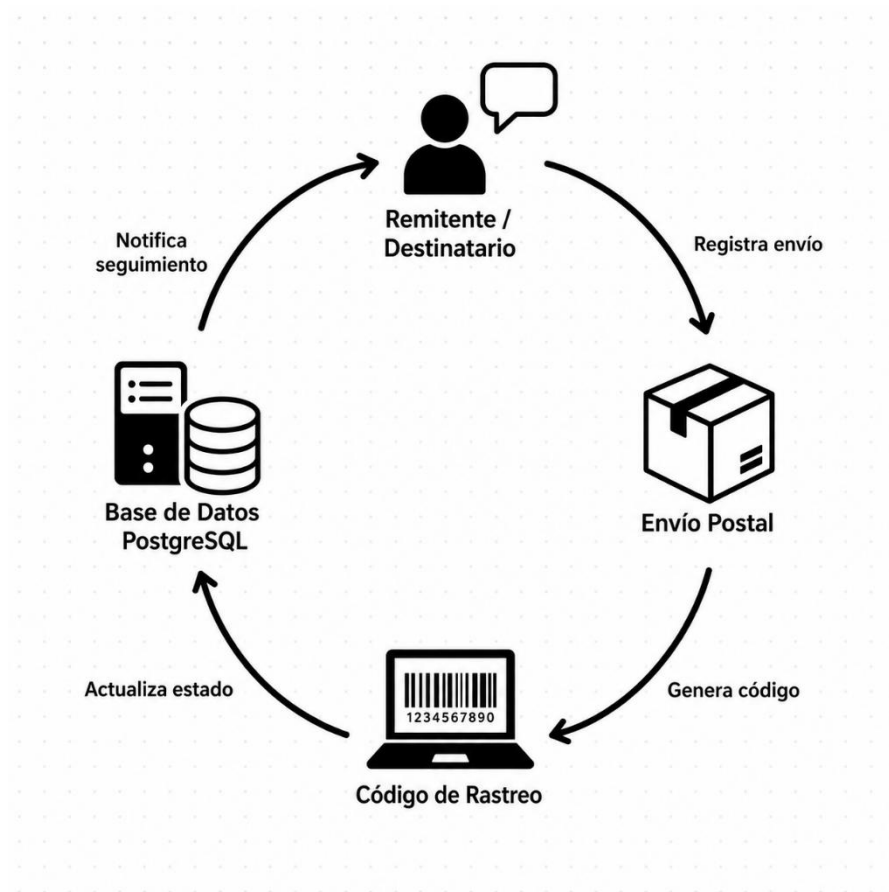
Se adoptó una arquitectura en capas combinada con una separación en servicios independientes. El sistema se divide en tres capas bien definidas: presentación (frontend React), lógica de negocio (backend Node.js/Express) y persistencia de datos (PostgreSQL). Cada capa se despliega como un servicio autónomo dentro de un entorno Docker Compose, lo que permite escalarlos o actualizarlos de forma independiente.

Justificación

Esta elección arquitectónica responde a varios criterios técnicos y prácticos:

- Separación de responsabilidades clara: el frontend se ocupa exclusivamente de la presentación, el backend de la lógica de negocio y la base de datos de la persistencia. Ninguna capa asume tareas que no le corresponden.
- Escalabilidad horizontal: si el volumen de envíos crece, es posible levantar múltiples instancias del backend sin tocar el frontend ni la base de datos.
- Mantenibilidad: los cambios en una capa no generan efectos colaterales sobre las demás, siempre que se respeten los contratos de la API.
- Facilidad de despliegue: el uso de Docker Compose permite replicar el entorno completo en cualquier máquina con un solo comando.

Diagrama de Arquitectura



5. Middleware y Sistemas Distribuidos

Middleware Utilizado

El sistema hace uso de varios componentes de middleware que intervienen en el procesamiento de cada petición HTTP:

- Express.js actúa como middleware principal de enrutamiento. Gestiona el ciclo de vida de cada request, delega la petición al controlador correspondiente y se encarga de serializar la respuesta en formato JSON.
- JWT (JSON Web Tokens) es el mecanismo de autenticación stateless del sistema. Cada vez que un usuario inicia sesión, el backend emite un token firmado que debe incluirse en las cabeceras de todas las peticiones protegidas. El middleware de autenticación verifica la firma del token antes de permitir el acceso a cada ruta privada.
- CORS (Cross-Origin Resource Sharing) es un middleware que habilita las peticiones entre el frontend (puerto 5173) y el backend (puerto 3001), dominios distintos bajo el mismo host en desarrollo local.
- Middleware de roles: además de verificar la identidad del usuario, existe una capa adicional que comprueba si el rol del usuario (admin o empleado) le da acceso a la operación solicitada. Por ejemplo, solo los administradores pueden crear o eliminar usuarios.
- Middleware de manejo de errores: captura excepciones no controladas en los controladores y devuelve respuestas de error consistentes con el formato esperado por el frontend.

Preparación para Sistemas Distribuidos

Aunque el sistema se despliega actualmente en un único host mediante Docker Compose, su arquitectura está pensada para evolucionar hacia un entorno distribuido. Las decisiones técnicas que lo facilitan son las siguientes:

- La API REST es completamente stateless: cada petición contiene toda la información necesaria para ser procesada, lo que permite distribuir el backend en múltiples nodos sin sincronización de sesión.
- La base de datos PostgreSQL es un servicio independiente. En el futuro puede migrarse a un cluster con réplicas de lectura o a un servicio gestionado en la nube sin modificar el backend.
- Se puede incorporar Redis como capa de caché para reducir la carga en consultas frecuentes, como la del portal de rastreo público.
- Para la gestión de notificaciones de cambio de estado, se podría integrar RabbitMQ u otro sistema de colas de mensajes, desacoplando el proceso de actualización del envío de su notificación al destinatario.
- A medida que el tráfico crezca, se puede agregar un API Gateway (como Kong o Nginx) para balanceo de carga, rate limiting y centralización de logs.

6. Decisiones Tecnológicas

Cada tecnología del stack fue seleccionada con una justificación técnica concreta, priorizando coherencia, rendimiento y facilidad de mantenimiento.

Tecnología	Categoría	Justificación
React + Vite	Frontend	React permite construir interfaces con componentes reutilizables y un manejo de estado eficiente. Vite reemplaza a webpack con tiempos de arranque y build significativamente más rápidos.
Node.js + Express	Backend	JavaScript full-stack reduce la curva de aprendizaje del equipo. Node.js es ideal para aplicaciones con muchas operaciones de I/O concurrentes gracias a su modelo asíncrono no bloqueante.
PostgreSQL	Base de Datos	Motor relacional robusto con soporte para transacciones ACID, consultas complejas y campos JSON. Es la mejor opción para modelar el historial de estados con integridad referencial.
Docker + Compose	Infraestructura	Garantiza que el entorno de desarrollo sea idéntico al de producción. Facilita la instalación en cualquier máquina y la futura migración a plataformas cloud.
Axios	HTTP Client	Librería de cliente HTTP para el frontend. Ofrece interceptores que permiten adjuntar el token JWT automáticamente en todas las peticiones salientes.
Recharts	Visualización	Librería de gráficos para React basada en SVG. Permite construir el dashboard gerencial con gráficos interactivos de forma declarativa.
Bcryptjs	Seguridad	Implementa el algoritmo bcrypt para el hashing de contraseñas, con soporte para factor de coste configurable.
Json2csv	Reportes	Conversión de objetos JSON a CSV en el backend, habilitando la exportación de reportes filtrados directamente desde la API.
Prettier	Tooling	Formateador de código automático que garantiza un estilo consistente en todo el proyecto, eliminando debates sobre convenciones.

7. Refactorización Aplicada

Durante el desarrollo se identificaron varios code smells que afectaban la legibilidad, mantenibilidad y coherencia del código.

Code Smell 1. Líneas partidas arbitrariamente (Long Lines / Inconsistent Formatting)

El código inicialmente presentaba saltos de línea en posiciones arbitrarias que no respondían a ninguna convención, lo que dificultaba su lectura y hacía que bloques lógicos como llamadas a funciones o condiciones se visualizaran fragmentados.

Mejora aplicada: se integró Prettier como herramienta de formateo automático con las siguientes reglas de configuración:

- `printWidth: 80` ninguna línea supera los 80 caracteres.
- `singleQuote: true` uso consistente de comillas simples en todo el proyecto.
- `tabWidth: 2` indentación con 2 espacios en lugar de 4.

El resultado fue un código más denso visualmente, donde cada línea tiene un propósito claro y los bloques lógicos se leen de corrido sin interrupciones innecesarias.

Code Smell 2. Funciones largas con responsabilidades mezcladas (Long Method)

El controlador de envíos concentraba en una sola función la validación de los datos de entrada, la construcción de la query SQL y la lógica de generación del código de rastreo. Esto violaba el principio de responsabilidad única y dificultaba tanto el testing como la reutilización de esa lógica.

Mejora aplicada: se desglosó la función en tres responsabilidades bien delimitadas:

- `validateEnvioInput(data)`: valida que todos los campos requeridos estén presentes y tengan el formato correcto.
- `generateTrackingCode(date)`: genera el código único con el formato ECO-YYYYMMDD-NNNNN a partir de la fecha del envío.
- `createEnvio(validatedData)`: ejecuta la inserción en base de datos y retorna el registro creado.

Esta separación redujo la longitud del controlador principal, facilitó la escritura de pruebas unitarias para cada función y permitió reutilizar `generateTrackingCode` en otros contextos del sistema.

Code Smell 3. Imports desordenados

Varios archivos del backend tenían bloques de imports organizados sin ningún criterio, lo que dificultaba identificar rápidamente las dependencias de cada módulo.

Mejora aplicada: se reordenaron los imports siguiendo una convención de tres grupos separados por línea en blanco: primero los módulos nativos de Node.js, luego las dependencias de terceros y finalmente los módulos internos del proyecto. Este orden, además de aplicarse manualmente, fue automatizado con Prettier y `eslint-plugin-import`.

8. Manual de Usuario

Acceso al Sistema

Abre un navegador y navega a <http://localhost:5173>. Se mostrará la pantalla de inicio de sesión. Ingresa tu correo electrónico y contraseña, y haz clic en Ingresar. Si las credenciales son correctas, serás redirigido al panel principal según tu rol.

Panel Principal (Dashboard)

El dashboard muestra un resumen del estado operativo actual: cantidad total de envíos por estado, gráficos de tendencia y alertas de envíos con retrasos. Es de solo lectura y se actualiza cada vez que se recarga la página.

Gestión de Envíos

- Para registrar un nuevo envío, haz clic en Nueva Envío dentro de la sección Envíos. Completa los datos del remitente, destinatario, sucursal de origen y destino, y selecciona el tipo de servicio. Al guardar, el sistema generará automáticamente el código de rastreo.
- Para actualizar el estado de un envío existente, búscalo usando los filtros disponibles (fecha, sucursal o estado), haz clic sobre él y selecciona el nuevo estado en el desplegable correspondiente.
- Puedes combinar varios filtros para acotar los resultados y exportarlos en CSV haciendo clic en Exportar CSV.

Portal de Rastreo Público

Sin necesidad de iniciar sesión, cualquier persona puede consultar el estado de un envío en <http://localhost:5173/rastrear>. Solo debe ingresar el código de rastreo en el campo indicado y presionar Buscar. Se mostrará el historial completo de estados con fechas y horas.

Administración (solo Administrador)

- En la sección Usuarios puedes crear nuevas cuentas, asignar roles y desactivar usuarios existentes.
- En la sección Sucursales puedes registrar o editar las sucursales del país, incluyendo nombre, ciudad y dirección.

Cierre de Sesión

Para salir del sistema, haz clic en el botón Salir ubicado en la esquina superior derecha de la barra de navegación. Esto invalida tu sesión local y te redirige a la pantalla de inicio.

9. Manual de Instalación

Sigue estos pasos para ejecutar el proyecto ECOBOL en cualquier equipo desde cero.

Requisitos Previos

- Docker Desktop instalado (Windows o macOS) o Docker Engine con Docker Compose (Linux).
- Git instalado para clonar el repositorio.
- Puertos 5173, 3001 y 5435 libres en el equipo.

Pasos de Instalación

1. Clona el repositorio con el comando: `git clone <url-del-repositorio>` y luego accede a la carpeta con: `cd ecobol`
2. Si el proyecto requiere un logotipo personalizado, coloca el archivo `logo.png` dentro de la carpeta `public` del frontend antes de continuar.
3. Desde la raíz del proyecto, ejecuta: `docker compose up --build`. Este comando descargará las imágenes necesarias, construirá los contenedores e inicializará la base de datos con los datos semilla.
4. Una vez que los contenedores estén corriendo, accede al frontend en `http://localhost:5173` y al backend en `http://localhost:3001`.

Credenciales Iniciales

Correo	Contraseña	Rol	Estado
<code>admin@ecobol.bo</code>	Admin123!	Administrador	Activo
<code>carlos.mamani@ecobol.bo</code>	Admin123!	Empleado	Activo

Acceso Directo a la Base de Datos

Si necesitas inspeccionar la base de datos directamente, puedes conectarte a PostgreSQL en `localhost:5435` con usuario `postgres` y contraseña `ecobol_pass`, usando cualquier cliente SQL como DBeaver o TablePlus.

Solución de Problemas Comunes

- Si algún contenedor no inicia, verifica que los puertos 5173, 3001 y 5435 no estén siendo usados por otro proceso.
- En Linux, asegúrate de que el usuario actual tenga permisos para ejecutar Docker sin `sudo`, o antepón `sudo` a los comandos docker.
- Si la base de datos no se inicializa correctamente, ejecuta `docker compose down -v` para eliminar los volúmenes y luego repite el comando de arranque.

10. Conclusiones

Aprendizajes del Proyecto

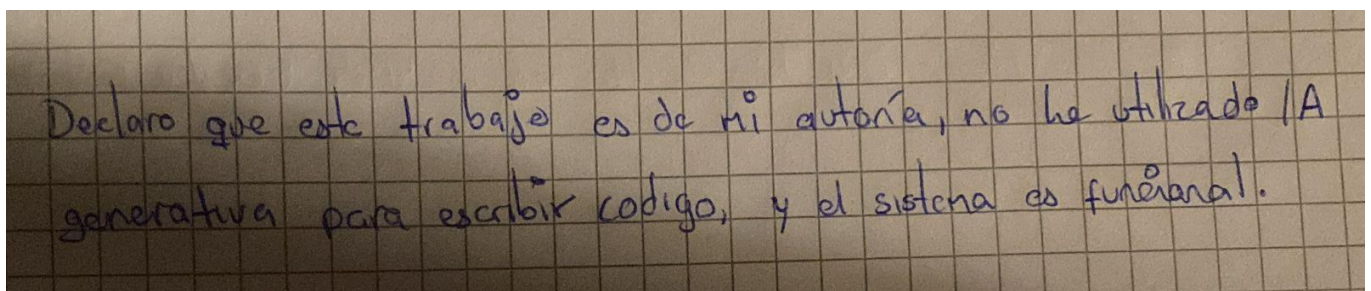
El desarrollo de este sistema dejó aprendizajes concretos en varios frentes. En lo arquitectónico, quedó demostrado que una separación clara de responsabilidades en capas independientes simplifica enormemente el mantenimiento y la evolución del sistema. En lo técnico, el uso de Docker Compose desde el inicio del proyecto eliminó los problemas de entorno que típicamente consumen tiempo en proyectos colaborativos. En lo metodológico, la aplicación de herramientas de formateo automático como Prettier demostró ser una inversión pequeña con un retorno alto en consistencia y legibilidad del código.

También fue valioso experimentar con la integración de autenticación JWT en un sistema full-stack, entendiendo los desafíos de gestionar tokens en el cliente, proteger rutas en el servidor y manejar la expiración de sesiones de forma transparente para el usuario.

Recomendaciones para ECOBOL

- Adopción gradual: implementar el sistema primero en una o dos sucursales piloto antes del despliegue nacional, para identificar ajustes operativos necesarios con menor riesgo.
- HTTPS en producción: antes de exponer el sistema a internet, debe configurarse un certificado SSL/TLS. Las credenciales y tokens JWT deben transmitirse siempre bajo conexión cifrada.
- CI/CD con GitHub Actions: establecer un pipeline de integración continua que ejecute tests y genere builds automáticos ante cada cambio en el repositorio, reduciendo el riesgo de regresiones.
- Notificaciones activas: en una segunda fase, integrar notificaciones por SMS o correo electrónico para informar automáticamente a los destinatarios cuando su envío cambie de estado.
- Capacitación del personal: el éxito del sistema depende de la adopción por parte de los empleados. Se recomienda diseñar una jornada de capacitación práctica antes de la puesta en producción.
- Auditorías de seguridad periódicas: revisar regularmente las dependencias del proyecto con herramientas como npm audit y actualizar aquellas con vulnerabilidades conocidas.

11. Declaración jurada



Declaro que este trabajo es de mi autoría, no he utilizado IA generativa para escribir código, y el sistema es funcional.